



二次开发接口

该章节介绍了二次开发接口的使用方法

版本号	修订日期	修订人	变更记录
1.0	2010.10.10	钟四化	初版
1.1	2010.12.01	杨秀锋	根据富士康需求，增加 Unicode、取消错误接口
1.2	2011.02.16	钟四化	根据定制部门的需求、增加对象及文档操作接口，提供更丰富的应用
1.3	2012.02.08	李仕勇	增加了运动控制接口

概述

本动态库基于 Hans Laser Marking Software V5.3以上版本进行开发，用于满足以下情况下一些高级用户的特殊需要。

- 1、打标系统只是客户系统的一部分，且客户具有较强的开发能力，希望将打标系统完全集成到他们自己的系统中。
- 2、海外非标准用户，由于存在沟通和出差的问题，使用这种方式可以有效地解决一些自动化应用的问题。
- 3、其它行业或中心可以基于本动态库开发他们自己的打标系统。

本动态库和打标软件一样，只支持 EMCC 卡。

本动态库采用和 Windows 相同的 API 方式导出接口，函数使用 stdcall 声明，可供 VC++、VB、Delphi、C++ Builder、C# 等多种开发工具使用，使用者需要显示调用，显示调用的方法如下：

- 1、调用 Windows API 函数 LoadLibrary()动态加载 DLL。
- 2、调用 WindowsAPI 函数 GetProcAddress()取得要调用函数的指针。
- 3、用函数指针调用 DLL 中函数完成相应功能。
- 4、程序结束或不再使用时，调用 WindowsAPI 函数 FreeLibrary()释放动态库。

软件接口动态库名：HansAdvInterface.dll

软件提供完整安装盘，按照安装步骤安装完成后，即可正常使用，在使用之前，需要进入标准软件进行相关参数的设置，如 BOX 校正、打标位置校正、激光器设置等。

注：软件接口和打标软件是不能同时使用的，如果打标软件正在使用，通过接口初始化设备时会返回错误码 1（另一个程序在运行），这种情况下需要关闭打标软件或校正软件。

返回值及结构定义

所有函数返回值均为一个整形的通用错误码，码值定义如下

```
#define HL_ERR_SUCCESS           0//成功
#define HL_ERR_PROGRUN          1//另一个程序在运行
#define HL_ERR_PATH              2//路径不正确
#define HL_ERR_FAILOPEN         3//初始化失败
#define HL_ERR_NOINITIAL        4//未初始化
#define HL_ERR_WARNING          5//报警
```

```

#define HL_ERR_OVERTIME           6//命令超时
#define HL_ERR_READFILE          7//无法读取文件
#define HL_ERR_NOFINDFONT        8//指定的字体不存在
#define HL_ERR_NOPEN             9//指定的层号不存在
#define HL_ERR_NOOBJECT          10//没找到指定的对象
#define HL_ERR_PARAM             11//参数非法
#define HL_ERR_ERROR_STATUS      12//错误的状态,当前状态下不能执行此操作
#define HL_ERR_ALLOCMEMORY       13//分配内存失败
#define HL_ERR_OVERRANGE         14//打标范围超限
#define HL_ERR_BUFFER_SIZE       15//缓冲区不够
#define HL_ERR_NULLPTR           16//空指针
#define HL_ERR_NODOC             17//未找到指定文档
#define HL_ERR_UNKNOWN           100//未知错误

```

```

#define WM_TOUCH_MSG              (WM_USER+ 1095) //主动检测脚踏触发消息

```

//填充参数结构体定义

```

struct baseHatch
{
    BOOL        bEnable;           //填充使能
    BOOL        bEquaDist;         //等间距使能
    BOOL        bKeepAngle;        //填充线绕轮廓旋转
    int         nFillStyle;         //填充类型
    double      dbSpace;           //填充间距
    double      dbAngle;           //填充角度
    double      dbBeamComp;         //光栅大小
    double      dbLineReduce;      //线缩进量
    double      dbStartOffset;      //起始缩进量
    double      dbEndOffset;        //末尾缩进量
    double      dbMinJump;          //最小跳转间距
    //
    BOOL        bLoop;             //缩扩使能
    int         nLoopNum;           //缩扩圈数
    double      dbLoopDist;         //缩扩间距
    BOOL        bRemoveLine;        //缩扩后去掉填充线
    BOOL        bRemoveOrg;         //缩扩后去掉原始轮廓

    int         nFillNum;           //填充次数
    double      ProAngle;           //累进角度
    double      AddDis;             //累进间距
    BOOL        bSaveCountour;      //累进填充时保留外框
} BaseHatch;

```

```

typedef struct HATCH_PARA
{
    BaseHatch    newFileData1;
    BaseHatch    newFileData2;
    BOOL         bFillFirst;
} HATCHPARA,*LP_HATCHPARA;

```

```

//字体参数结构体定义
typedef struct FONT_PARA
{
    int      nFontType;           //字体类型0-TTF, -SHX, -HANS, -DOT
    BYTE     nCharSet;           //字符集,Unicode编码时用
    char     szFontName[100];    //TTF、HANS字体用
    char     szShxCN [100];      //SHX、DOT字中文字体用
    char     szShxEN[100];       // SHX、DOT字英文字体用
    char     szShxNum [100];
} FONTPARA,*LP_FONTPARA;

```

```

//层参数结构体定义
typedef struct LAYER_PARA
{
    int      nCount;             //打标次数
    double   dbMarkV;            //矢量打标速度(mm/s)
    double   dbJumpV;           //矢量空跳速度(mm/s)
    double   dbQFre;             //Q频(KHz)
    double   dbQRIs;             //Q释放(us)
    double   dbCurrent;          //电流(A)
    int      nFpkTime;           //首脉冲抑制时间
    double   dbFPKInitRate;      //首点比例(0.0~1.0)
    double   dbFPKLenRate;       //抑制长度系数(0.0~0.999)
    double   dbFPKMaxPRate;      //峰值能量比例(0.0~1.0)
    double   dbFPKCurRate;      //首脉冲电流系数(0.0~1.0)
    int      nQuality;           //打标质量系数
    int      nLayerDelay;        //层延时
    int      nLaserOnDelay;      //激光开延时(us)
    int      nLaserOffDelay;     //激光关延时(us)
    int      nMoveLineDelay;     //走笔延时(us)
    int      nJumpDelay;         //跳转延时(us)
    int      nRoundDelay;        //拐弯延时(us)
    double   dbCurrentR;         //电流(%)
    double   dbSimmerCur;       //维持电流(%)
    int      nWaveForm;          //波形
    double   dbPowerR;           //功率(%)
} LAYERPARA,*LP_LAYERPARA;

```

接口函数说明

A、通用部分，所有用户均可使用

1、HS_GetDllVersion

语法：int HS_GetDllVersion(WORD* pMainVer,WORD *pDllVer)

作用：获取 Dll 版本及其所需主程序版本信息，版本信息由三段组成，共16位，从高到低分别分配 4、5、7位，如主程序版本号为 20869,二进制排列为 0101 00011 0000101，解析结果为 V5.3.5。

参数描述：

pMainVer：主版本号，DLL 所依附主程序的版本号，必须安装此版本的程序 DLL才能正常工作。

pDllVer：子版本号，DLL 内部版本号，区分不同的 DLL。

2、HS_InitialMachine

语法：int HS_InitialMachine(LPCTSTR pszPath)

作用：初始化控制系统

参数描述：库文件所在路径，pszPath 为空指针时取内部默认路径

3、HS_CloseMachine

语法：int HS_CloseMachine()

作用：关闭控制系统

参数描述：无

4、HS_GetMarkRange

语法：int HS_GetMarkRange(double *pX,double *pY);

作用：获取打标范围。

参数描述：

pX：Box X 方向大小(mm)。

pY：Box Y 方向大小(mm)。

5、HS_LoadMarkFile

语法：int HS_LoadMarkFile(LPCTSTR pszFileName)

作用：加载打标文件,如果成功,自动设置该文档为当前工作文档。

参数描述：

pszFileName：打标文件完整路径及名称。

6、HS_GetDocList

语法：int HS_GetDocList(TCHAR szDocList[][50],int* pCount)

作用：获取当前所有已打开文档列表,文档名不允许超过 50Bytes。

参数描述：

szDocList：文档列表指针。

pCount：传入缓冲区数组大小，处理完成后返回实际获取文档数。

7、HS_SetCurDoc

语法：int HS_SetCurDoc(LPCTSTR lpszDocName);

作用：设置当前工作文档。

参数描述:

lpszDocName: 文档名称 (不含后缀.HS 和路径)。

8、HS_SaveMarkFile

语法: int HS_SaveMarkFile(LPCTSTR lpszFile)

作用: 保存打标文件,需要传入完整路径及文件名。

参数描述:

lpszFile: 完整路径及文件名, 如果为 NULL,则按文件打开时的路径及文件名保存。

9、HS_CloseMarkFile

语法: int HS_CloseMarkFile(LPCTSTR lpszFile,BOOL bSave)

作用: 保存及关闭打标文档。

参数描述:

lpszFile: 文档名称 (不含后缀.HS 和路径)。

bSave: 关闭时是否保存当前文档,按原文件保存。

[注]默认文档不允许关闭,只能关闭通过接口打开的文档。

10、HS_IsTouch

语法: int HS_IsTouch(int* pTouchFlag)

作用: 上位机查询检测脚踏, 检测效率较低, 间隙较短时可能会漏检。

参数描述:

pTouchFlag: 信号标志位, 1-触发, 0-未触发

注: 硬件设置中脚踏部分设置对此接口有效, 如触发电平、几个信号触发一次等。

11、HS_CheckTouch

语法: int HS_CheckTouch(HWND hWnd,BOOL bActive)

作用: 快速脚踏检测, 由下位机实时检测, 一旦检测到脚踏信号, 会向指定窗口发 TOUCH消息, 可以检测到很短间隙的信号, 高端需求时用。

参数描述:

hWnd: 信号触发后, 接收 WM_TOUCH_MSG 消息的窗口

bActive: 激活或关闭主动检测。

注: 关闭主动检测后可用 HS_IsTouch 或 HS_ReadPort 检测 TOUCH 信号。

12、HS_Mark

语法: int HS_Mark(int nType,BOOL bWaitTouch,BOOL bWaitEnd, int
nOverTime,BOOL bMarkAll)

作用: 打标

参数描述:

nType 打标类型 0-正常打标, 1-红光预览

bWaitTouch 是否等待脚踏信号, TRUE-检测到信号后才打标, FALSE-立即打标

bWaitEnd 是否阻塞, TRUE-打标结束后才返回, FALSE-不阻塞, 立即返回

nOverTime 等待脚踏信号超时时间(ms)

bMarkAll 是否全部打标(TRUE-全部, FALSE-选择部分)

13、HS_IsMarkEnd

语法: int HS_IsMarkEnd(int* pFlag)

作用: 检测打标是否结束 (HS_Mark 非阻塞调用时)。

参数描述: pFlag 打标标志, 0-正在打标, 1-打标正常结束, 2-打标中止结束, 3-设备异常打标结束

14、HS_MarkPause

语法: int HS_MarkPause()

作用: 打标暂停

参数描述: 无

15、HS_Remark

语法: int HS_Remark()

作用: 暂停后恢复打标

参数描述: 无

16、HS_MarkStop

语法: int HS_MarkStop()

作用: 中止打标

参数描述: 无

17、HS_GetMarkTime

语法: int HS_GetMarkTime(UINT *pMarkTime);

作用: 获取最后一次打标时间。

参数描述:

pMarkTime: 打标时间(ms)

18、HS_ChangeTextByName

语法: int HS_ChangeTextByName(LPCTSTR lpszTextName, LPCTSTR lpszText)

作用: 改变文本内容

参数描述:

lpszTextName: 可变文本别名

lpszText: 文本内容

19、HS_ChangeTextByNameW

语法: int HS_ChangeTextByNameW(LPCTSTR lpszName, LPCWSTR
lpszTextValue)

作用: 改变文本内容(UNICODE 编码)

参数描述:

lpszTextName: 可变文本别名

lpszText: 文本内容

20、HS_GetTextByName

语法: int HS_GetTextByName(LPCTSTR lpszTextName, LPTSTR lpszText, int*
pMaxCount)

作用: 获取可变文本内容

参数描述:

lpszTextName: 可变文本别名

lpszText: 接收文本内容的缓冲区

pMaxCount: 接收缓冲区的字节数, 如果接收长度超过缓冲区大小, 该参数返回接收内容的长度。

注: 如果缓冲区太小, pMaxCount 返回实际需要的大小。

21、HS_GetTextByNameW

语法: int HS_GetTextByNameW(LPCTSTR lpszName, LPWSTR lpszText, int* pMaxCount)

作用: 获取可变文本内容 (UNICODE 编码)

参数描述:

lpszTextName: 可变文本别名

lpszText: 接收文本内容的缓冲区

pMaxCount: 接收缓冲区的字节数; 如果接收长度超过缓冲区大小, 该参数返回接收内容的长度。

注: pMaxCount 返回实际需要的缓冲区大小

22、HS_ReadPort

语法: int HS_ReadPort(DWORD* pValue)

作用: 获取 IO 输入状态

参数描述:

pValue: 输入口状态值

23、HS_WritePort

语法: int HS_WritePort(DWORD dwValue, DWORD dwMask)

作用: 输出 IO 信号

参数描述:

dwValue: 输出信号值

dwMask: 有效信号掩码, 掩码位为 1 的信号才输出, 0xFFFFFFFF 输出全部信号

24、HS_SetDevCfg

语法: int HS_SetDevCfg()

作用: 弹出系统设置对话框, 设置硬件参数

参数描述: 无

25、HS_PreviewGraph

语法: int HS_PreviewGraph(HWND hWnd, double dbCenterX, double dbCenterY, double dbScale)

作用: 预览当前打标内容

参数描述:

hWnd: 在指定的窗口中显示打标内容

dbCenterX: 显示中心点 X 坐标, 默认0

dbCenterY: 显示中心点 Y 坐标, 默认0

dbScale: 预览缩放系数, 默认 1.0, 大于 1 时放大显示, 小于 1 时缩小显示。

26、HS_SetHatchPara

语法: int HS_SetHatchPara(LPVOID pPara)

作用: 设置填充参数

参数描述:

pPara: 填充参数数据指针, 见 HATCHPARA 定义

27、HS_GetHatchPara

语法: int HS_GetHatchPara(LPVOID pPara)

作用: 获取当前填充参数

参数描述:

pPara: 填充参数数据指针, 见 HATCHPARA 定义

28、HS_SetFontPara

语法: int HS_SetFontPara(LPVOID pPara)

作用: 设置字体参数

参数描述:

pPara: 字体参数数据指针, 见 FONTPARA 定义

29、HS_GetFontPara

语法: int HS_GetFontPara(LPVOID pPara)

作用: 获取当前字体参数

参数描述:

pPara: 字体参数数据指针, 见 FONTPARA 定义

30、HS_SetLayerPara

语法: int HS_SetLayerPara(int nLayer, LPVOID pPara)

作用: 设置指定层参数

参数描述:

nLayer: 当前要设置层层号。

pPara: 层参数数据指针, 详见 LAYERPARA 定义。

31、HS_GetLayerPara

语法: int HS_GetLayerPara(int nLayer, LPVOID pPara)

作用: 获取指定层参数信息

参数描述:

nLayer: 当前要获取层层号。

pPara: 层参数数据指针, 详见 LAYERPARA 定义。

32、HS_ImportVectorFile

语法: int HS_ImportVectorFile(LPCTSTR lpszFileName, double dbPosX,double
dbPosY,double dbWidth, double dbHeight, double
dbRotate,BOOL bHatch,int nHatchLayer, LPCTSTR
lpszObjName)

作用: 导入矢量图形文件

参数描述:

lpszFileName: 导入文件名称,通过后缀名识别类型(Plt、DXF)

dbPosX,dbPosY: 导入后图形放置位置

dbWidth: 导入后图形缩放大小

dbHeight: 导入后图形缩放宽度

dbRotate: 导入后图形旋转角度

bHatch: 是否填充

nHatchLayer: 填充层号

lpszObjName: 对象名称

33、HS_ImportImageFile

语法: int HS_ImportImageFile(LPCTSTR lpszFileName, double dbPosX,double
dbPosY,double dbWidth,double dbHeight,double
dbRotate,int nMarkType, double dbDotDistX,double
dbDotDistY, int nAccDot,BOOL bReciprocate, int
nRepairDot,LPCTSTR lpszObjName)

作用: 导入位图图形文件

参数描述:

pszFileName: 导入文件名称,通过后缀名识别类型
dbPosX,dbPosY: 导入后图形放置位置
dbWidth: 导入后图形缩放大小
dbHeight: 导入后图形缩放宽度
dbRotate: 导入后图形旋转角度
nMarkType: 位图打标类型 0-灰度位图, 1-二值位图
dbDotDistX: 位图X点距
dbDotDistY: 位图Y点距

以下参数只有打标类型为灰度位图有效,二值位图无效

nAccDot: 加速点个数
bReciprocate: 往复扫描
nRepairDot: 补偿点个数
lpszObjName: 对象名

注: 由于灰度位图打标参数调节很复杂,因此灰度位图的调节需要通过打标软件进行调节,库内部会调用已经调好的参数,不再额外提供开发接口。

34、HS_AddText

语法: int HS_AddText(LPCTSTR lpszStr, double dbHeight, double
dbWidthRatio, BOOL bModel, LPCTSTR pszName, double
dbPosX, double dbPosY, int nAlign, double dbAngle, int
nLayer, BOOL bHatch, int nHatchLayer, LPCTSTR lpszObjName)

作用: 添加文本数据

参数描述:

pszStr: 文本内容
dbHeight: 文本高度
dbWidthRatio: 宽度系数
bModel: 是否是可变文本
nModelID: 可变文本ID
dbPosX: 文本X位置
dbPosY: 文本Y位置
nAlign: 排列方式
dbAngle: 旋转角度
nLayer: 边框层号
bHatch: 是否填充
nHatchLayer: 填充层号
lpszObjName: 对象名

35、HS_AddText2

语法: int HS_AddText2(LPCTSTR lpszStr, double dbHeight, double
dbWidthRatio, BOOL bModel, LPCTSTR pszName, double
dbPosX, double dbPosY, int nAlign, double dbAngle, int
nLayer, BOOL bHatch, int nHatchLayer, double dbCharSpace,
double dbLineSpace, LPCTSTR lpszObjName)

作用: 添加文本数据,和HS_AddText相比,增加字间距和行间距参数

参数描述:

pszStr: 文本内容
dbHeight: 文本高度

dbWidthRatio: 宽度系数
 bModel: 是否是可变文本
 nModelID: 可变文本ID
 dbPosX: 文本X位置
 dbPosY: 文本Y位置
 nAlign: 排列方式
 dbAngle: 旋转角度
 nLayer: 边框层号
 bHatch: 是否填充
 nHatchLayer: 填充层号
 dbCharSpace: 字符间距
 dbLineSpace: 行间距
 lpszObjName: 对象名

36、HS_AddBarcode

语法: int HS_AddBarcode(LPCTSTR lpszStr, int nBarType, double dbHeight, double
 dbPosX, double dbPosY, double dbAngle, int nAlign, int
 nLayerNo, double dbBarRatio, double dbLineDist, double
 dbShrink, BOOL bReverse, double dbRevWidth, double
 dbRevHeight, BOOL bModel, LPCTSTR lpszTextName,
 LPCTSTR lpszObjName)

作用: 添加条码数据

参数描述:

pszStr: 条码内容
 nBarType: 条码类型
 0 - EAN8,
 1 - EAN13,
 2 - EAN128,
 3 - CODE128A,
 4 - CODE128B,
 5 - CODE128C,
 6 - CODE128AUTO,
 7 - CODE39,
 8 - INT25,
 9 - EXT39,
 10 - UPCA,
 11 - UPCE,
 12 - PDF417,
 13 - DATAMATRIX
 14 - QR

dbHeight: 条码高度
 dbPosX: 条码X位置
 dbPosY: 条码Y位置
 dbAngle: 旋转角
 nAlign: 排列方式
 nLayerNo: 层号
 dbBarRatio: 宽窄比
 dbLineDist: 线间距

dbShrink: 缩进量
bReverse: 条码是否反填
dbRevWidth: 反填宽度 (mm
dbRevHeight: 反填高度 (mm,为时不反填
bModel: 是否是可变文本
nModelID: 可变文本ID
lpszObjName: 对象名

37、HS_AddCurve

语法: int HS_AddCurve(double ptBuf[][2],int nPtNum,int nLayer,BOOL bHatch,int
nHatchLayer, LPCTSTR lpszObjName)

作用: 添加曲线数据

参数描述:

ptBuf: 曲线点数组
nPtNum: 点个数
nLayer: 使用层号
bHatch: 是否填充
nHatchLayer: 填充层号
lpszObjName: 对象名

38、HS_AddDotMatrix

语法: int HS_AddDotMatrix(double ptBuf[][2],int nPtNum,int nLayer, LPCTSTR
lpszObjName)

作用: 添加点阵数据

参数描述:

ptBuf: 曲线点数组
nPtNum: 点个数
nLayer: 使用层号
lpszObjName: 对象名

39、HS_GetObjList

语法: int HS_GetObjList(TCHAR szObjList[][50],int* pCount)

作用: 获取当前工作文档中对象列表。

参数描述:

szObjList: 对象名称列表指针。
pCount: 传入缓冲区数组大小, 处理完成后返回实际获取对象数。

40、HS_SelectObjects

语法: int HS_SelectObjects(LPCTSTR lpszObjName, BOOL bSelected)

作用: 选择/不选择指定对象, 选中对象后, 可对选中对象进行打标、移动、旋转、切变、阵列、提取属性、删除等操作。

参数描述:

lpszObjName: 对象名称, 如果为 NULL, 则对全部对象进行操作。
bSelected: 是否选择, TRUE-选择, FALSE-不选择。

41、HS_ClearObjects

语法: int HS_ClearObjects()

作用: 删除当前选中对象。

参数描述: 无

42、HS_Move

语法: int HS_Move(double dbMoveX,double dbMoveY)

作用: 对当前选中对象进行移动操作。

参数描述:

dbMoveX: X移动量(mm)。

dbMoveY: Y移动量(mm)。

43、HS_Rotate

语法: int HS_Rotate(double x,double y,double dbRotate)

作用: 对当前选中对象进行旋转操作。

参数描述:

x: 旋转基准点 X 坐标。

y: 旋转基准点 Y 坐标。

dbRotate: 旋转角度 (度)

44、HS_Shear

语法: int HS_Shear(double dbBaseX,double dbBaseY, double dbAngleX,double dbAngleY)

作用: 对当前选中对象进行切变操作。

参数描述:

dbBaseX: X 轴切变位置。

dbBaseY: Y 轴切变位置。

dbAngleX: X 切变角度(度)。

dbAngleY: Y 切变角度(度)。

45、HS_Scale

语法: int HS_Scale(double x,double y,double dbScaleX,double dbScaleY)

作用: 对当前选中对象进行缩放操作。

参数描述:

x: 基准点 X 坐标。

y: 基准点 Y 坐标。

dbScaleX: X 缩放系数。

dbScaleY: Y 缩放系数。

46、HS_Array

语法: int HS_Array(double ptArray[][2],int nSize)

作用: 对当前选中对象进行阵列操作。

参数描述:

ptArray: 阵列位置坐标数组。

nSize: 阵列数组大小。

47、HS_GetObjProperty

语法: int HS_GetObjProperty(double* pLeft,double *pTop,double *pRight,double *pBottom)

作用: 获取当前选中对象的属性, 返回对象 Quad[l,t,r,b]的信息。

参数描述:

pLeft: Quad left 坐标

pTop: Quad top 坐标。

pRight: Quad right坐标。

pBottom: Quad bottom 坐标。

48、HS_IgotoXYZ

语法: int HS_IgotoXYZ(int nType, double dbX,double dbY, double dbZ)

作用: 立即指令, 移动 XYZ 轴到指定位置 (XY-指振镜, Z 指三维调焦)

参数描述:

nType: (类型 0-只移动 XY, 1-只移动 Z, 2-三轴同时移动)

dbX: (X 移动目标位置坐标,mm)

dbY: (Y 移动目标位置坐标,mm)

dbZ: (Z 移动目标位置,范围 0--65535)

49、HS_MoveMotorV

语法: int HS_MoveMotorV(int nAxisNo, double dbMoveValue, int nMoveModel,
int nMoveType)

作用: 电机单轴运动,

参数说明:

nAxisNo: 轴号(运动轴号, 通过轴号确定运动方式为旋转/平移)

dbMoveValue: 运动量(旋转运动时为度, 平台运动时为mm)

nMoveModel: 运动模式: 0绝对 1相对 2回机械原点

nMoveType: 运动类型: 0带负载运动 1空程运动

50、HS_HomeMoveV

语法: int HS_HomeMoveV(int nAxisNo, BOOL bSoftHome)

作用: 电机单轴回零

参数说明:

nAxisNo: 轴号

bSoftHome: 是否回软件原点

51、HS_SetAsSoftHome

语法: int HS_SetAsSoftHome(int nAxis)

作用: 将当前位置设为软件原点

参数说明:

nAxis: 轴号

52、HS_ILaserOn

语法: int HS_ILaserOn(double dbQFrq,double dbQRls,double dbCurrentA,double
dbPower)

作用: 立即指令, 开激光

参数描述:

dbQFrq: Q 频,

dbQRls: Q 释放时间

dbCurrentA: 激光工作电流(A), 调 Q 类激光器用。

dbPower: 激光工作功率(%),IPG、SPI 激光器用。

53、HS_ILaserOff()

语法: int HS_ILaserOff()

作用: 立即指令, 关激光

参数描述: 无

54、HS_GetLastError

语法: int HS_GetLastError(int* pError, LPTSTR lpszMsg, int nSize)

作用: 获取最近一次错误码及其详细错误信息

参数描述:

pError: 错误码

lpszMsg: 保存错误信息的缓冲区

nSize: 错误信息缓冲区大小

```
*****
*      B、高级应用，部分应用需要授权      *
*****
```

高级应用需要的数据定义:

//对外接口点数据结构

```
class CInterDot
{
public:
    CInterDot()
    {
        x = 0;
        y = 0;
        z = 0;
    }
    CInterDot(double x1,double y1)
    {
        x = x1;
        y = y1;
    }

    CInterDot(double x1,double y1,double z1)
    {
        x = x1;
        y = y1;
        z = z1;
    }

    CInterDot & operator=(const CInterDot &dot)
    {
        if ( this == &dot )
            return *this;
        else
        {
            x = dot.x;
            y = dot.y;
            z = dot.z;
        }
        return *this;
    }
}
```

```

public:
    double x; //x
    double y; //y
    double z; //z (3D预留)
};

//对外接口线条数据类
class CInterStroke: public CObject
{
public:
    CInterStroke()
    {
        m_nLayer = 0;
        m_dotArray.RemoveAll();
    }

    virtual ~CInterStroke ()
    {
        m_dotArray.RemoveAll();
    }

    void AddDot(CInterDot& dot){
        m_dotArray.Add(dot);
    }

    void SetLayer( int nLayer ){
        m_nLayer = nLayer;
    }

    CInterStroke & operator=(const CInterStroke &stroke)
    {
        if( this == &stroke )
            return *this;
        else
        {
            m_nLayer = stroke.m_nLayer;
            m_dotArray.RemoveAll();
            m_dotArray.Append(stroke.m_dotArray);
        }
        return *this;
    }
public:
    CArray<CInterDot, CInterDot>m_dotArray;
    int m_nLayer;
};

//对外接口线条数据链表
typedef CTypedPtrList<CObList, CInterStroke *>InterStrokeList;

```

55、HS_GetStrokeList

语法: int HS_GetStrokeList(LPCTSTR lpszObjName,LPVOID pList)

作用: 获取当前指定对象的线条数据, DLL 内部分配内存, 外部用完后用HS_FreeStrokeList释放。

参数说明:

lpszObjName: 指定对象名称, NULL 时取全部对象。

pList: 存放提取的线条数据, 详见 InterStrokeList 定义。

56、HS_MarkStorkeList

语法: int HS_MarkStorkeList(int nType,LPVOID pList)

作用: 打标传入的 InterStrokeList 对象线条数据。

参数说明:

nType: 0-激光, 1-红光

pList: 线条数据。

57、HS_FreeStorkeList

语法: int HS_FreeStorkeList(LPVOID pList)

作用: 释放自定义线条数据。

参数说明:

pList: InterStrokeList线条数据。

58、HS_SendCustomCmd [*]

语法: int HS_SendCustomCmd(*****, *****)

作用: 上位机主动和下位机通讯, 详见内部使用文档。

[注]: 本接口仅供内部使用, 不对外开放。

59、HS_SetCustomMsgCall [*]

语法: typedef void (*HS_USERDEFINE_MSG_CALLBACK)(*****);

int HS_SetCustomMsgCall(HS_USERDEFINE_MSG_CALLBACK pCallBack);

作用: 下位机主动向上位机通讯用。

[注]: 本接口仅供内部使用, 不对外开放。